NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC2006

Software Engineering

Team Members:

No	Name	Matriculation Number
1.	CHIN WEI HAO	U2322704F
2.	SU JING LONG, BRIAN	U2221634D
3.	MURUGESAN NUVAAN	U2421726K
4.	BIAN LINGZHU	U2422772F
5.	Gupta Hitansh	U2423473J
6.	YU ZHENGWEN	U2322264J

1. Introduction **3**
1.1 Purpose 3
1.2 Data Dictionary 3
1.3 References 5
2. Overall Description **6**
2.1 Product Perspective 6
2.2 Product Functions 6
2.3 User Classes and Characteristics 6
1. Primary User – Driver (General User) 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 7
2.6 Assumptions and Dependencies 7
3. Functional Requirements **8**
3.1. User Login 8
3.2. User Registration 8
3.3. Tabbed Navigation Buttons 8
3.4. Main View 8
4. Nonfunctional Requirements **10**
4.1. Performance 10
4.2. Scalability 10
4.3. Security 10
4.4. Reliability 11
4.5. Usability 11
5. Use case diagram & description **11**
5.1 Use case diagram 11
5.2 Use case description 12
6. UI Mockup **31**
6.1 User login 31
6.2 User registration 32
6.3 Map 33
6.4 My Parking 34
6.5 Profile 36
7. Dialogue Map **38**
8. Entity Class Diagram **39**
9. Stereotype Class Diagram **39**

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a detailed description of the functional and non-functional requirements of CarParkSmart. This document serves as a reference for developers, testers, and project stakeholders to understand the intended behaviour and constraints of the system.

This SRS specifies the requirements for a mobile application that assists users in finding nearby car parks with the lowest parking rates, tracking parking charges, and managing parked vehicle locations. It also defines features that support parking reminders, family-based car location sharing, and navigation assistance.

The scope of this document is limited to the software components of the CarParkSmart application. External systems such as public car park data APIs, mapping services, and mobile operating system services are considered dependencies and are not specified in detail within this document.

1.2 Data Dictionary

Term	Definition
User	An individual who uses the CarParkSmart application.
Username	A unique customizable identifier for each User
Car	A car registered by a user for parking tracking purposes.
Car Name	A user defined name for a registered car
Carpark	A designated parking facility available to users.

Carpark Name	A name of the Carpark e.g. Blk 285
Carpark Location	The geographical coordinates of a car park.
Parking Rate	The cost charged for parking at a car park.
Parking Session	A period during which a vehicle is parked in a car park.
Parking Start Time	The time when a parking session begins.
Parking End Time	The time when a parking session ends.
Parking Session Driver	Driver who created the parking session
Estimated Fee	The estimated parking cost incurred by an active parking session
Accumulated Fee	The total parking cost incurred during a parking session.
Rate Alert	A notification triggered when parking fees exceed a user-defined limit.
Rate Alert Threshold	The maximum parking fee set by the user before an alert is issued.
Parked Car Location	The stored location of the vehicle after parking.

Car Location Update	A manual update of the parked vehicle's location by the user.
Car Location Photo	An image reference to help users locate their parked vehicle.
Family Group	A group of users allowed to share parking location information.
Shared Access	Permission granted to family members to view the last parked location.
Navigation Assistance	Directions provided to guide users back to their parked vehicle.
Public Carpark API	An external data source that provides car park locations and rates.

1.3 References

Public Carpark API

- Carpark availability: https://data.gov.sg/datasets/d_ca933a644e55d34fe21f28b8052fac63/view
- Carpark rates:
- https://data.gov.sg/datasets/d_9f6056bdb6b1dfba57f063593e4f34ae/view

2. Overall Description

2.1 Product Perspective

Singapore has a car owner rate of almost 40%. Many families and companies share cars and use it as their main mode of transport. Users often face challenges locating affordable parking, tracking and remembering their parking costs, as well as finding where the car was parked after finishing their activity. Existing navigation or car park finders do not adequately address these parking needs.

2.2 Product Functions

Locate nearby car parks and identify the cheapest available option based on parking rate data obtained from public APIs.

- Allow users to confirm when they have parked and initiate a parking session.
- Track parking duration and estimate parking charges using published car park rates.
    - Notify users when parking charges exceed a user-defined rate threshold.
- Allow users to manually record and update the parked car location.
    - Store photo references to assist users in locating their parked vehicle.
- Provide navigation guidance from the user's current location to the parked car location.
- Enable family members to view the last known parked location of a shared vehicle.
- Manage parking history and display past parking records for registered vehicles.

2.3 User Classes and Characteristics

1. Primary User – Driver (General User)

- Uses the application regularly to locate nearby car parks and find the cheapest parking options.
- Can be working adult or student, with basic knowledge of technology usage
- Initiates and manages parking sessions by confirming when the vehicle is parked.
- Tracks parking rates and receives alerts when costs exceed predefined thresholds.
- Manually records and updates parked car locations.
- Uses navigation and photo references to locate the parked vehicle.
- Has basic technical proficiency with mobile applications.

Most functional requirements in this SRS are primarily designed to satisfy this user class.

2.4 Operating Environment

- The software will mainly be implemented in mobile platforms, mainly Android and iOS devices.
- For our demo, we will be focusing on creating a working prototype for Android phones.
- The application will require smartphones with standard mobile hardware components, including GPS, Internet Access, Touchscreen interface
- For features such as the fee alert, it will require being able to run as a background process

2.5 Design and Implementation Constraints

Due to limitations of a mobile device, we are unable to easily and reliably detect when a user has parked. As such, the system will require manual user input to indicate the start of a parking session.

- The application must comply with constraints imposed by the Android and iOS operating systems, including permissions for location access and background processing.
- The system is constrained by the capabilities and availability of external Public Carpark APIs, which provide parking rate and location data.

2.6 Assumptions and Dependencies

Public Carpark APIs provide accurate and reasonably up-to-date parking rate and location information.

- Users grant the necessary permissions (e.g. location access) required for the system to function correctly.
- The system will also depend on third-party map and navigation services and APIs for displaying maps and providing navigation assistance.

3. Functional Requirements

3.1. User Login

3.1.1. The system shall allow a user to log in to the application using Google OAuth 2 Authentication

3.1.2. The system will redirect user back to the app after logging into Google

3.2. User Registration

3.2.1. The system shall allow a user to create an account

3.2.1.1. The system will create a new user using Google OAuth 2 Authentication

3.2.1.1.1. The system will redirect user back to the app after logging into Google

3.2.1.1.2. The system will prompt the user to add a Username and profile picture during profile setup

3.2.1.1.2.1. The Username shall be longer than 5 characters

3.2.1.1.2.2. The Username shall not be longer than 30 characters

3.2.1.1.2.3. The Username must be unique, else the system displays a custom error message “Username already exists” and prompts re-entry.

3.2.1.1.2.4. Adding a profile picture shall be optional

3.3. Tabbed Navigation Buttons

3.3.1. The system shall allow the user to switch between ‘Map’, ‘My Parking’ and ‘Profile’ Tabs

3.4. Main View

3.4.1. Map

3.4.1.1. The map shall show the user’s current location in the center upon loading

3.4.1.2. The map shall identify and display the nearby carparks available

3.4.1.2.1. Tapping the Carpark shall display the Parking Rate and Availability

3.4.1.2.2. The nearby carparks data shall be retrieved from the Public Carpark API

3.4.2. ‘Start Parking Session’ Action Button

3.4.2.1. The button shall allow the user to create a new parking session via pop up page

3.4.2.1.1. The page shall allow users to choose which car is assigned for this parking session

3.4.2.1.2. The page shall show the user their current parked location

3.4.2.1.2.1. The page should allow the user to edit their location precisely if necessary

3.4.2.1.3. The page shall display the Carpark details

3.4.2.1.3.1. The details shall include Parking Rate, Carpark Name and Location taken from public carpark API

3.4.2.1.4. The page shall allow the user to set session name and description

3.4.2.1.5. The page shall be able to add relevant pictures via a dialog

3.4.2.1.5.1. The dialog shall allow users to upload pictures from gallery

3.4.2.1.5.2. The dialog shall allow users to take a picture directly using their phone camera

3.4.2.1.6. The page shall allow the user to set a Rate Alert Threshold

3.4.2.1.6.1. The app shall trigger a Rate Alert by creating a notification when the Estimated Cost exceed the Threshold

3.4.2.1.7. The page shall contain a start parking button to start the session

3.4.2.1.7.1. The system shall provide default metadata for any session initiated without a user-defined name or description

3.4.2.2. The new parking session shall be viewable in **My Parking Page**

3.4.3. My Parking Page

3.4.3.1. The page shall show a list of cars registered under the user

3.4.3.1.1. Each entry shall be expandable to show the list of Parking Sessions based on the chosen car

3.4.3.2. List of Parking Sessions

3.4.3.2.1. The list of parking sessions shall show up to 3 most recent entries displayed in reverse chronological order.

3.4.3.2.2. Each entry shall show the Session Name, Carpark Name

3.4.3.2.3. Tapping the list entry shows the Parking Start Time, Parking End Time, Accumulated Fee or Estimated Cost, Session Driver, User-Uploaded Pictures and Carpark Location

3.4.3.2.3.1. The entry shall have stop session button if the session has not been ended

3.4.3.2.3.2. Tapping the Carpark Location shall redirect the user to their navigation app (eg: Google Maps) with the Carpark Location as destination

3.4.3.2.3.3. Tapping the pictures shall show a popup with an enlarged picture

3.4.4. Profile Page

3.4.4.1. Personal Information

3.4.4.1.1. The system shall allow a user to view their profile information

3.4.4.1.1.1. The profile information shown shall include their Username, Google email and profile picture

3.4.4.1.2. The system shall allow a user to delete their account

3.4.4.2. Family Management

3.4.4.2.1. The system shall allow the user to create a family

3.4.4.2.1.1. Each account shall only be able to create 1 family

3.4.4.2.1.2. Each family created shall have a join code

3.4.4.2.2. The system shall allow the user to join a family using a join code

3.4.4.2.3. The system shall display the list of family members if the user is in a family

3.4.4.2.4. The system shall allow a user to generate a join code for the family group membership.

3.4.4.2.4.1. The family group membership shall have a limit 5

3.4.4.2.5. The system shall allow a user to remove other users from the family group membership

3.4.4.2.6. The system shall display all entries and cars registered under each individual user to all the members in the family group

3.4.4.3. Car Management

3.4.4.3.1. The system shall display the list of existing cars registered by the user

3.4.4.3.1.1. The details displayed shall include Carplate, Car Name, and a preset car icon.

3.4.4.3.2. The system shall allow a user to add a new car by clicking the add button

3.4.4.3.2.1. The button shall direct to the user to a pop up page

3.4.4.3.2.2. The User shall fill in the Carplate, Car Name and choose from a preset car icon

3.4.4.3.2.3. If any details are missing the system shall prompt the user with an error message that shows “not all required fields are filled in”.

4. Nonfunctional Requirements

4.1. Performance

4.1.1. The application must have a start up time of under 3 seconds

4.1.2. The main view map data should be loaded under 5 seconds upon successful login

4.1.3. The Carpark data should be loaded under 5 seconds upon selecting the Carpark

4.1.4. The Parking Session list shall be loaded and displayed under 3 seconds upon clicking on ‘My Parking’

4.1.5. The Parking Session data shall be loaded and displayed under 3 seconds upon clicking on the entry

4.1.6. After a parking session is created, it should be recorded into the backend system under 5 seconds

4.1.7. The system shall update parking fee calculations at intervals not exceeding 1 minute during an active parking session.

4.1.8. After joining a family via a join code, the system should add the user into the family and display all members under 10 seconds

4.2. Scalability

4.2.1. The application must handle up to 500 simultaneous users without a degradation in performance

4.2.2. The system shall maintain responses time within defined performance threshold when the number of active users increase

4.3. Security

4.3.1. Users of the application must be authenticated using secure OAuth2 Protocol

4.3.2. All sensitive information listed in **3.4.4** shall be encrypted.

4.3.3. All user data listed in **3.4.4** shall be deleted when their account is deleted

4.3.4. The system shall restrict shared parking data listed in **3.4.3** to authorized family group members only.

4.3.5. The system shall automatically log out users after 15 minutes of inactivity

4.4. Reliability

4.4.1. The application shall maintain a crash free session rate of at least 99% over a 30 day period

4.4.2. The system shall display a notification stating that “There is an error retrieving third party data” if external API data is unavailable

4.4.3. The system shall automatically retry failed API requests up to 5 attempts before displaying an error message.

4.4.4. The system shall maintain data consistency when network connectivity is temporarily lost and synchronize data once connectivity is restored.

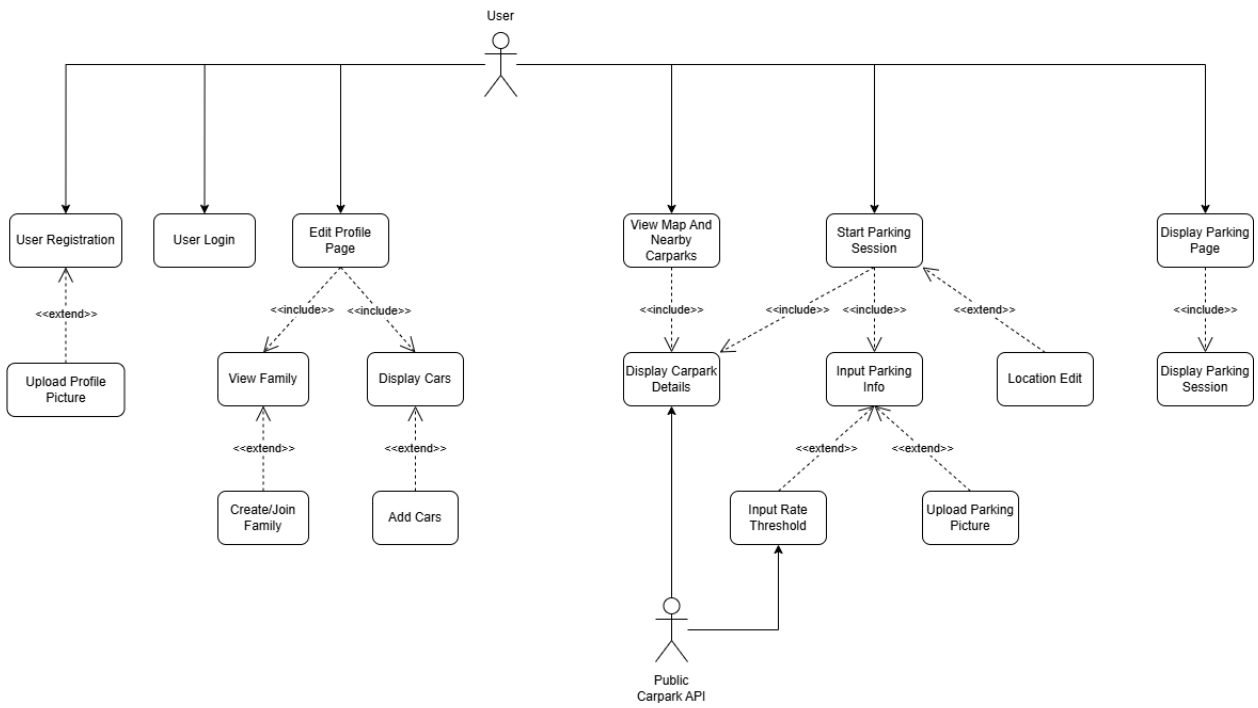
4.5. Usability

4.5.1. Help messages must be displayed in the local language according to the user’s locale.

4.5.2. Users shall navigate between pages using the application's icons in under 3 seconds.

5. Use case diagram & description

5.1 Use case diagram



5.2 Use case description

Use Case ID:	001		
Use Case Name:	UserLogin		
Created By:	Bian Lingzhu	Last Updated By:	Bian Lingzhu
Date Created:	10/2/2026	Date Last Updated:	30/3/2026

Actor:	User
Description:	Allows a user to create a new account using Google OAuth 2 Authentication and complete profile setup
Preconditions:	1. User has a valid email or Github account
Postconditions:	2. User has logged in successfully 3. User is redirected back to the app
Priority:	High
Frequency of Use:	High
Flow of Events:	1. If user wants to login with GitHub, user selects the “Sign in with GitHub” button 2. System redirects user to GitHub authorization page 3. User logs in to GitHub and grants permission to the application

	4. System redirects user to the application 5. If user wants to login with Magic Link, user selects the “Sign in with Magic Link” button 6. User inputs an email to log in with 7. System sends a log in link to the user’s email 8. User clicks the log in link and is redirected to the application 9. Upon successful login, user is directed to the Map tab
Alternative Flows:	None
Exceptions:	1. If user cancels authentication, system redirects user back to login screen without logging in
Includes:	None
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

//////////////

	6. User enters their username and uploads a profile picture with use case **UploadProfilePicture** 7. System validates username length and uniqueness 8. System creates an account and redirect user back to application's main screen
Alternative Flows:	None
Exceptions:	1. If any required information is not inputted, display an error message to notify user 2. If username is too short (<5 characters), system displays a custom error message "Username needs to be over 5 characters" and prompts re-entry. 3. If username already exists, system displays a custom error message "Username already exists" and prompts re-entry.
Includes:	UploadProfilePicture
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

Alternative Flows:	AF-S1: User chooses take a picture from camera 1. Application will use the phone camera to take a picture 2. Continue to step 3 AF-S2: User chooses chooses upload from gallery 1. User can choose a past picture to upload from the phone's photo gallery 2. Continue to step 3 AF-S3: User chooses to not upload a profile picture 1. A default picture will be inserted as the profile picture 2. Continue to step 5
Exceptions:	1. If camera/gallery permission denied, system displays permission request message "Enable camera/gallery permissions to upload picture" 2. If uploaded file format not supported or image file is too large, system displays custom error message of upload failure: "File format not supported"
Includes:	None
Special Requirements:	Enable access to camera and/or photo gallery
Assumptions:	None
Notes and Issues:	None

//////////////

	2. System loads map centered on User's current location 3. System retrieves nearby carparks from Public Carpark API and displays all carparks nearby on the map 4. System will load more carparks based on where the user drags the map to
Alternative Flows:	AF-S1: If user taps a Carpark 1. Details of it are displayed with use case **DisplayCarparkDetails** 2. User can tap the back button to return to step 2 AF-S2: User can drag map around to observe nearby carparks between steps 3 and 4
Exceptions:	1. If location permission is denied, system displays permission request message "Enable location permissions to locate nearby carparks" 2. If no nearby carparks are found, system displays a pop-up error message "No nearby carparks found" to inform user
Includes:	DisplayCarparkDetails
Special Requirements:	Location permission enabled Nearby carparks retrieved from Public Carpark API
Assumptions:	None
Notes and Issues:	None

////////////

	3. System will display the Carpark Name, Parking Rate, availability and address of the carpark
Alternative Flows:	AF-S1: User can scroll to view all the information on the displayed page
Exceptions:	1. If selected carpark data fails to load, system displays retry option
Includes:	None
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

	10. System creates parking session
Alternative Flows:	AF-S1: User wants to edit their location to be more precise 1. User manually edits location with the **LocationEdit** use case 2. Continue to step 4 AF-S2: User wants to view the carpark's specific details before choosing it 1. System displays the carpark's details with use case **DisplayCarparkDetails** 2. User can go back to step 2
Exceptions:	1. If the user does not have a registered car, system displays error message "Add a car first to start a new parking session", prompting user to add a car first 2. If user already has an ongoing parking session, system prevents creation and prompts to end existing session 3. If any required information is not inputted, display an error message "Some required information is missing" to notify user
Includes:	LocationEdit DisplayCarparkDetails InputParkingInfo
Special Requirements:	Location permission enabled
Assumptions:	None
Notes and Issues:	None

////////////

Postconditions:	1. Location of the parking spot has successfully been changed
Priority:	Medium
Frequency of Use:	Low
Flow of Events:	1. User selects the “edit location” button 2. System displays the map and a pin icon, which is at the current carpark’s location 3. User drags the pin icon around the map to adjust the displayed location to be more precise 4. User confirms the final location displayed on the map
Alternative Flows:	None
Exceptions:	1. If map fails to load, system displays retry option
Includes:	None
Special Requirements:	None
Assumptions:	User is still parking at the same carpark, and parking rates will not change with this edit
Notes and Issues:	None

//////////////

	5. System prompts user to input their preferred rate alert threshold 6. User sets a rate alert threshold with use case **InputRateThreshold** 7. User can continue creating their parking session
Alternative Flows:	None
Exceptions:	1. If any required information is not inputted, display an error message “Some required information is missing” to notify user
Includes:	InputRateThreshold UploadParkingPicture
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

	5. Notification will show the current estimated cost, as well as the user's threshold
Alternative Flows:	AF-S1: User does not input any threshold 1. System will not record any threshold 2. No notification will be sent to the user during this parking session
Exceptions:	1. If rate threshold is not a positive decimal number, system displays an error message "Please input a valid number" 2. If notifications are denied, system displays permission request message "Enable notifications permissions to receive updates and alerts" 3. If notification permissions are rejected, system will record the threshold but not send notifications to the user (tbc)
Includes:	None
Special Requirements:	None
Assumptions:	Notifications are enabled
Notes and Issues:	None

//////////////

	3. System displays a preview of the parking picture and prompts for confirmation 4. User presses confirm 5. User returns to creating their parking session
Alternative Flows:	AF-S1: User chooses take a picture from camera 1. Application will use the phone camera to take a picture AF-S2: User chooses chooses upload from gallery 1. User can choose a past picture to upload from the phone's photo gallery AF-S3: User chooses to not upload a parking picture 1. No picture will be displayed and operations continue as normal
Exceptions:	1. If camera/gallery permission denied, system displays permission request message "Enable camera/gallery permissions to upload picture" 2. If uploaded file format not supported or image file is too large, system displays custom error message of upload failure: "File format not supported"
Includes:	None
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

Flow of Events:	1. User navigates to My Parking tab with navigation buttons 2. System displays a list of their registered cars in the page 3. Each car can expand to show their recent parking sessions, showing session name and carpark name 4. User can interact with each parking session with **DisplayParkingSession** use case
Alternative Flows:	None
Exceptions:	1. If no registered cars are found, display an empty page
Includes:	DisplayParkingSession
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

//////////////

	2. Navigation map successfully gives directions to the carpark location, and helps with further navigation AF-S2: User taps the picture of the parking session 1. Will show a popup with an enlarged picture
Exceptions:	1. If no valid navigation app is installed, display custom error message “No valid navigation app is installed, redirect unsuccessful”
Includes:	None
Special Requirements:	None
Assumptions:	For AF-S1: If user has used manual location adjustment, the app still redirects user based on the carpark location, not the manually adjusted location
Notes and Issues:	None

////////////

	2. User can click onto their username and profile picture to edit them, and choose to finalise the information 3. User will confirm the information inputted with the “Save Changes” button 4. System will save the information, showing a loading icon at the “Save Changes” button 5. System will redirect user back to the Profile page AF-S2: If user selects the “Cars” button, trigger the **DisplayCars** use case AF-S3: If user selects the “Family” button, trigger the **ViewFamily** use case AF-S4: If user selects the “Delete Account” button to delete their account 1. System will prompt confirmation for deletion 2. User must confirm deletion 3. Account will be deleted AF-S5: If user selects the “Logout” button, gets redirected to Login page
Exceptions:	1. If profile data failed to load, display retry option 2. If account deletion fails, displays an error message “Account deletion failed, please try again” 3. If any information is not edited, save the information as its previous state
Includes:	ViewPersonalInfo DisplayCars ViewFamily
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

Date Created:	10/2/2026	Date Last Updated:	30/3/2026

Actor:	User
Description:	Allows User to view who is in their family
Preconditions:	1. User is logged in 2. User is displaying their profile page
Postconditions:	1. Able to view people in the family if user is in a family 2. Able to continue to create or join a family if user is not currently in a family
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User taps on “Family” button 2. If user is in a family, system will display the family management page, with a list of names of family members 3. System will also display a join code at the top of the page
Alternative Flows:	AF-S1: User not in a family 1. The family management page will display a message “You are not in a family” 2. User can create or join a new family with the **CreateFamily** or **JoinFamily** use case AF-S2: If user is the owner of the family, he can delete the family with the “delete family” button 1. All members of the family will leave the family after family deletion AF-S3: If user is a member of the family, he can leave the family with the “leave family” button AF-S4: If user is the owner of the family, he can edit the family name 1. User presses an edit icon besides current family name 2. System displays a pop-up with a textbox for user to edit current family name 3. User presses save button to confirm the current family name

Exceptions:	1. If system fails to generate a unique join code, system displays a retry option
Includes:	CreateFamily JoinFamily
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

////////////

////////////

Use Case ID:	017		
Use Case Name:	DisplayCars		
Created By:	Bian Lingzhu	Last Updated By:	Bian Lingzhu
Date Created:	10/2/2026	Date Last Updated:	19/2/2026

Actor:	User
Description:	Allows user to view the cars that they or their family have registered
Preconditions:	1. User is logged in 2. User is displaying their profile page
Postconditions:	1. All cars are displayed in a list
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User selects the “Cars” button 2. System displays all registered cars by themselves and their family members 3. Each car displayed will include Carplate, Car Name, and a preset car icon. 4. User can select the “add car” button to add a new car, where the **AddCar** use case is triggered
Alternative Flows:	None
Exceptions:	1. If no cars are registered, display an empty state message 2. If system failed to retrieve car list, display a retry option
Includes:	AddCar
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

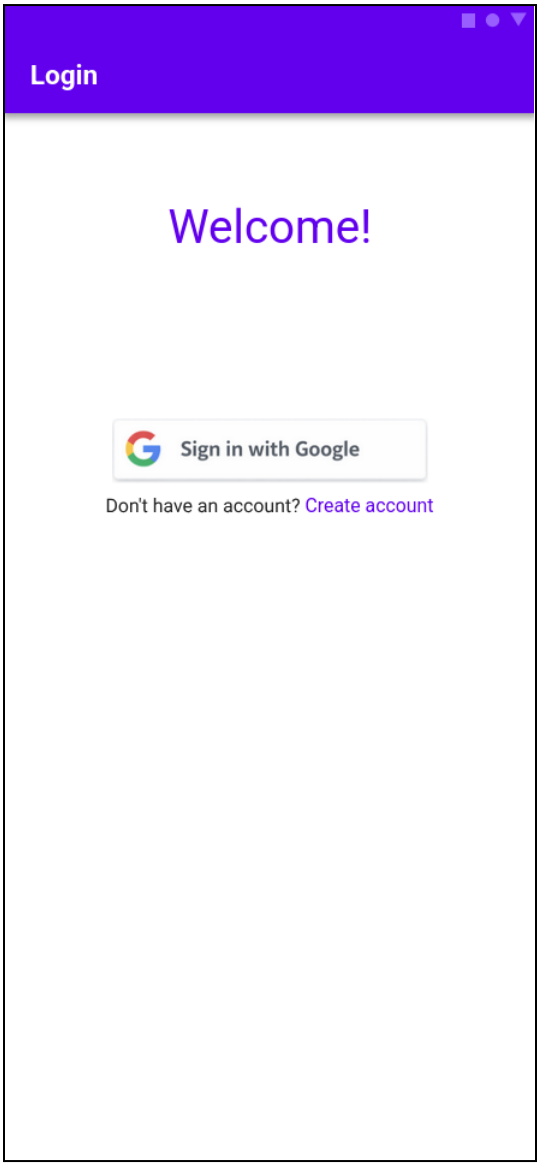
//////////////

Actor:	User
Description:	Allows user to add a new car
Preconditions:	1. User is logged in 2. User is viewing their cars
Postconditions:	1. All cars are displayed in a list
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User selects the “add car” button 2. System directs user to a pop up page to input details about the car 3. User inputs the Carplate, Car Name and choose from a preset car icon 4. System prompts user to confirm the information 5. User presses confirm to save the new car details
Alternative Flows:	None
Exceptions:	1. If any required information is not inputted, display an error message “Some required information is missing” to notify user 2. If the carplate number is in an invalid format, display an error message “Invalid carplate number, please enter a new carplate number 3. If carplate already exists in the system, display an error message to inform user
Includes:	None
Special Requirements:	None
Assumptions:	A valid carplate number follows the format of Singapore’s Vehicle Registration Number (VRN)
Notes and Issues:	None

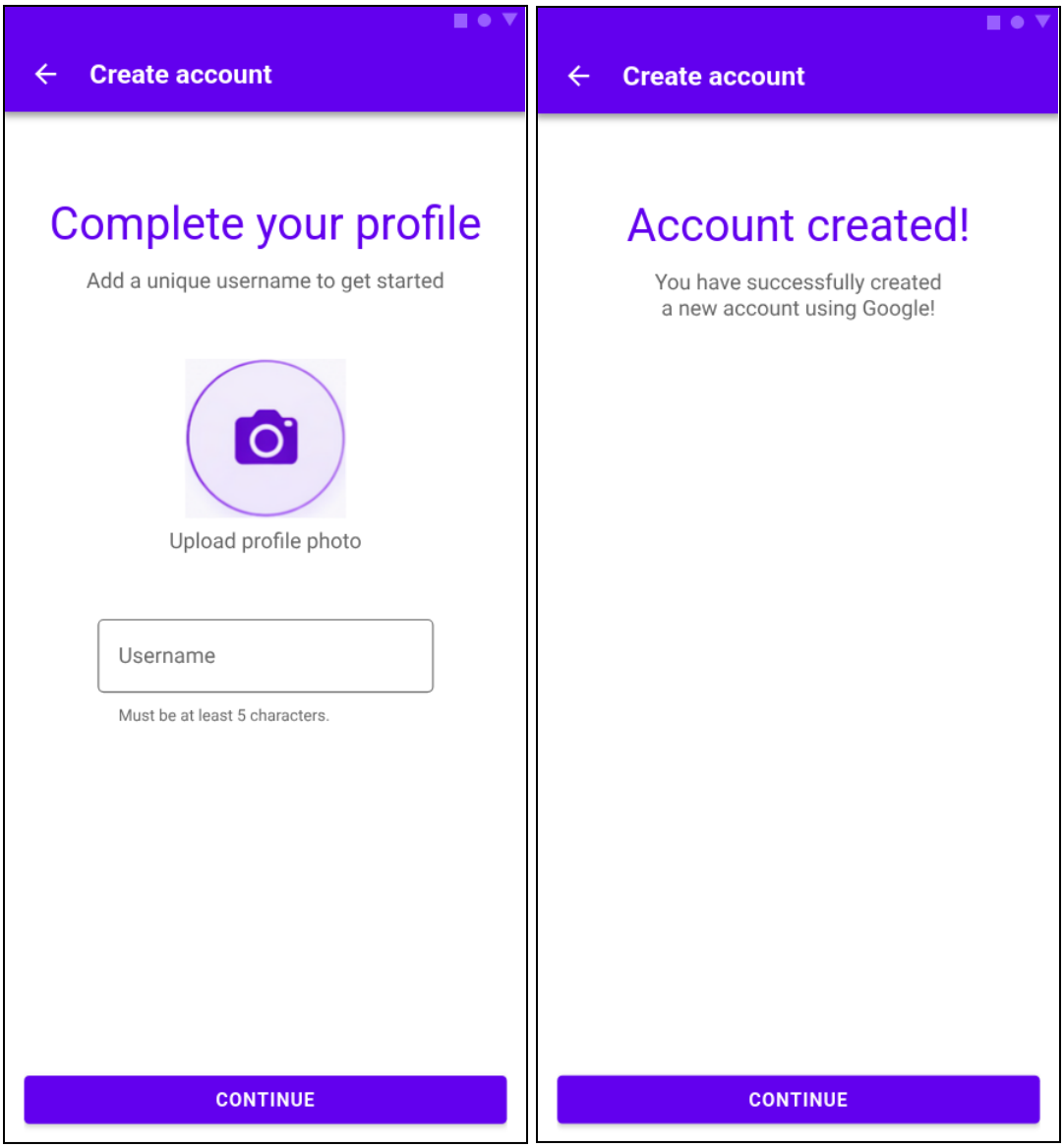
////////////

6. UI Mockup

6.1 User login

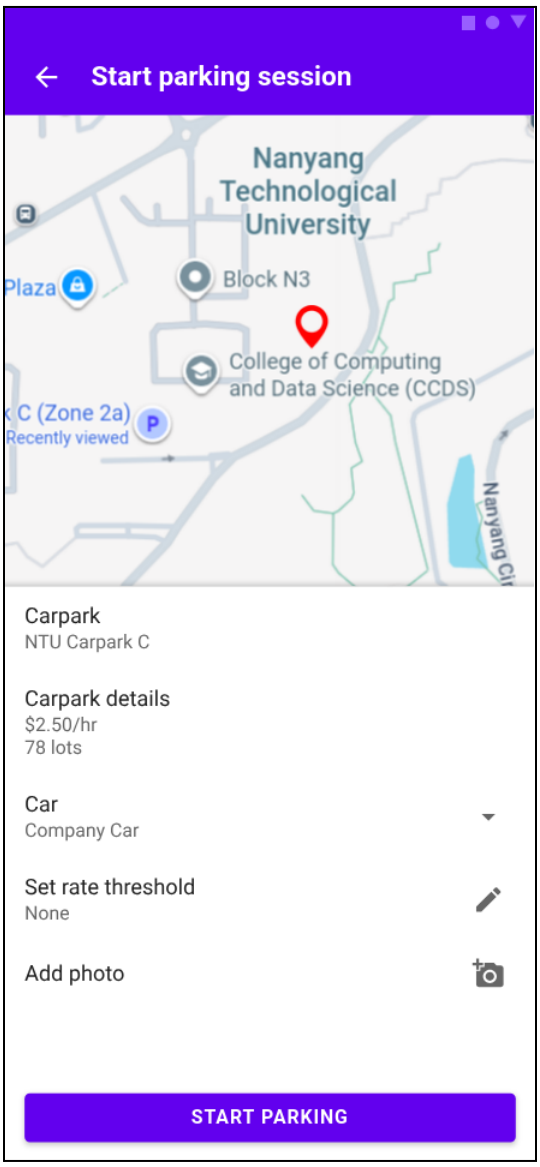


6.2 User registration

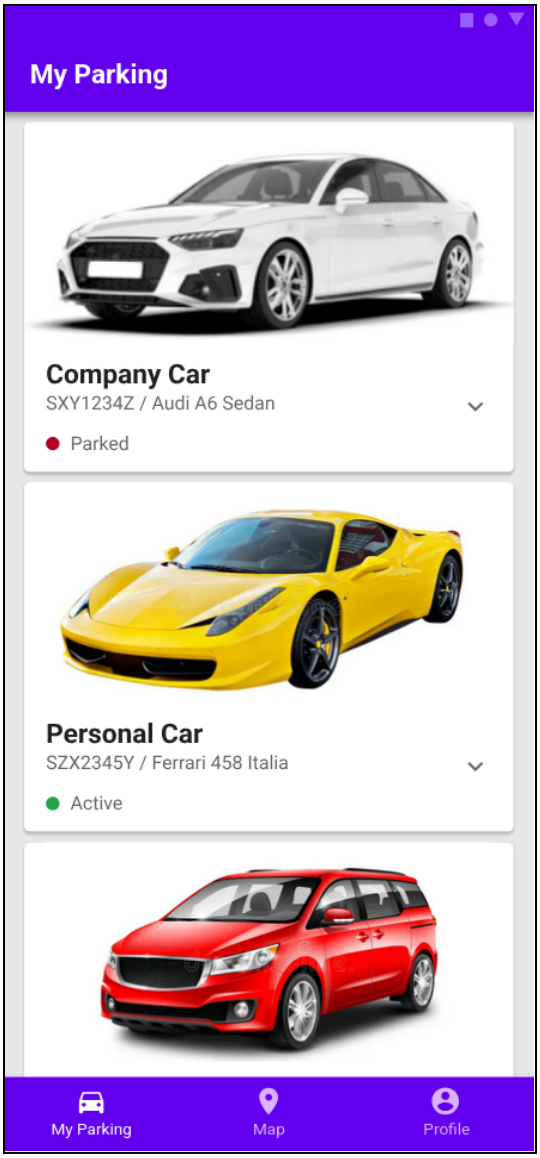


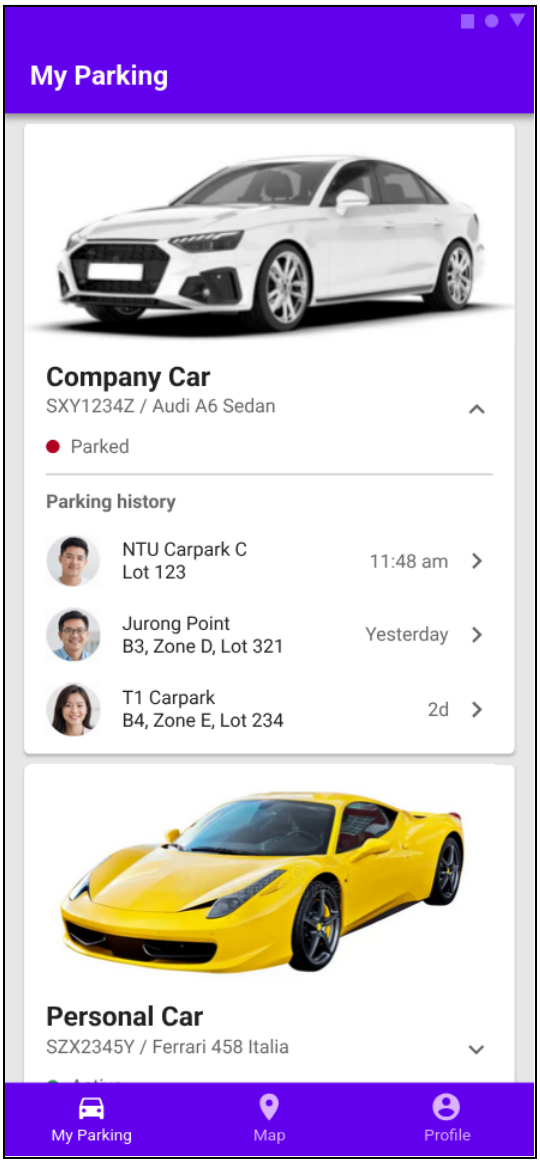
6.3 Map

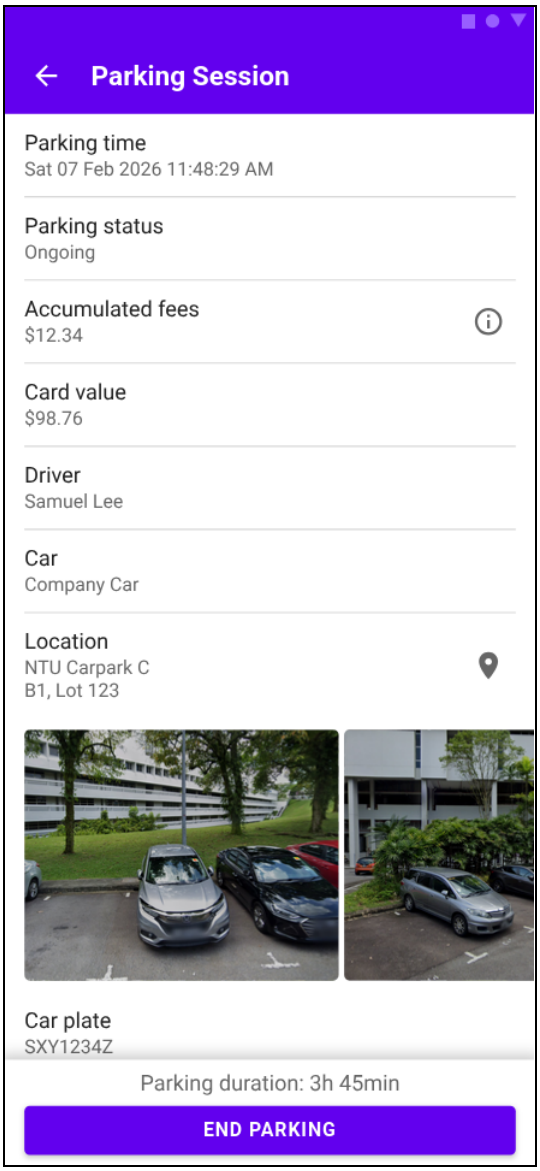
Map
Carpark B
$2.00/hr
56 lots
Carpark C
$2.50/hr
78 lots
Nanyang
Technological
University
Block N3
College of Computing
and Data Science (CCDS)
Carpark A
$1.50/hr
34 lots
The Hive
(Learning Hub
South - LHS)
PARK NOW
My Parking
Map
Profile

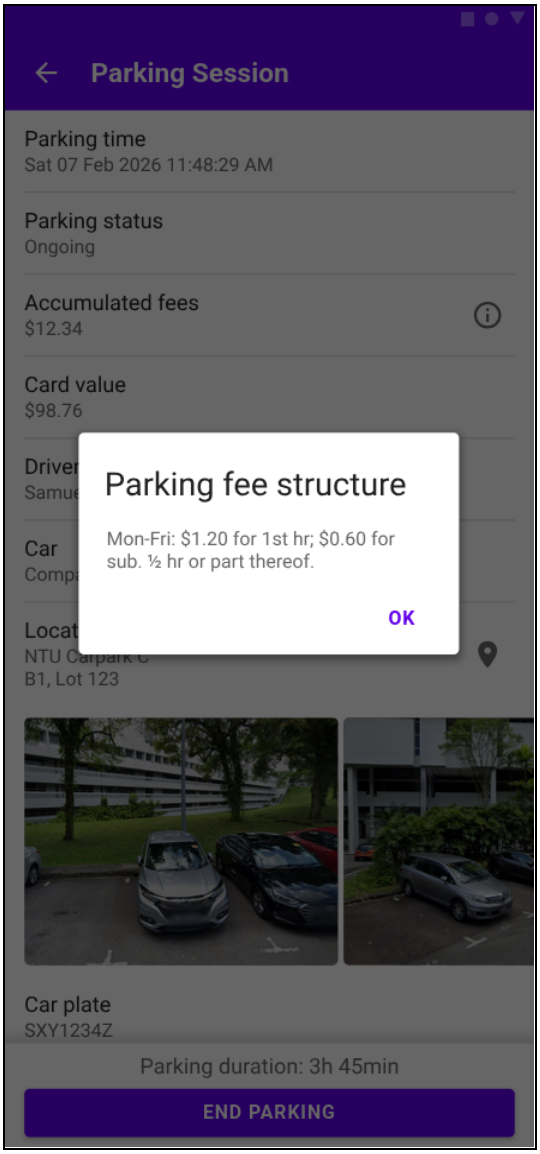

Start parking session
Nanyang
Technological
University
Block N3
College of Computing
and Data Science (CCDS)
Carpark
NTU Carpark C
Carpark details
$2.50/hr
78 lots
Car
Company Car
Set rate threshold
None
Add photo
START PARKING

6.4 My Parking

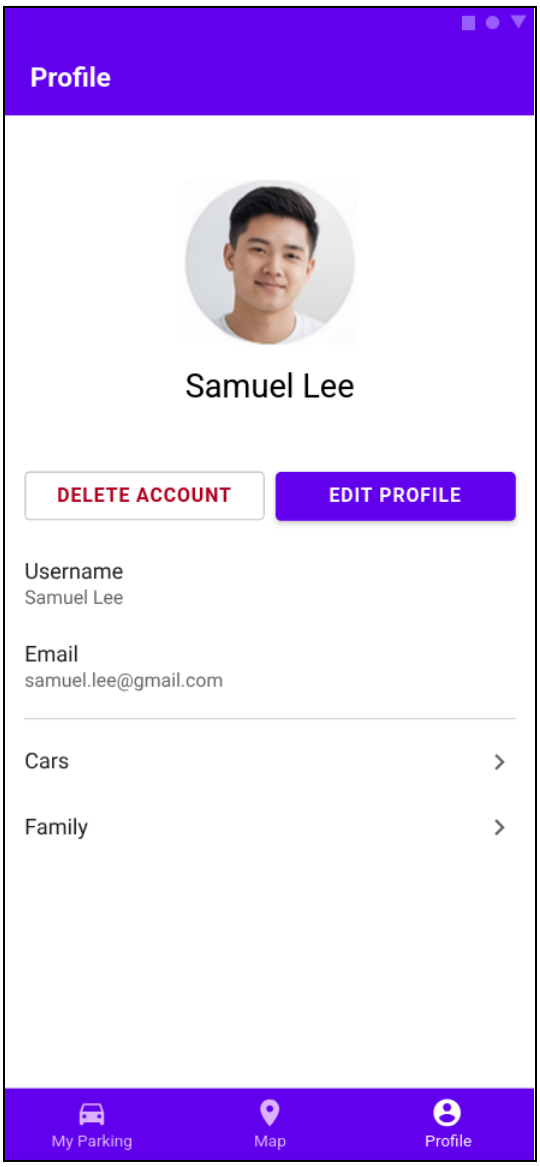


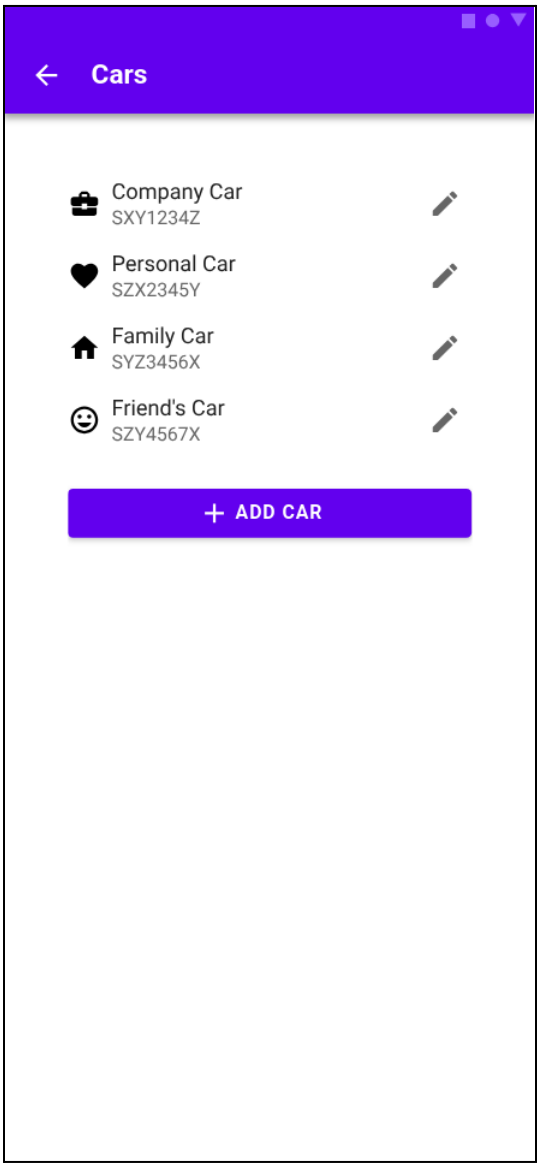


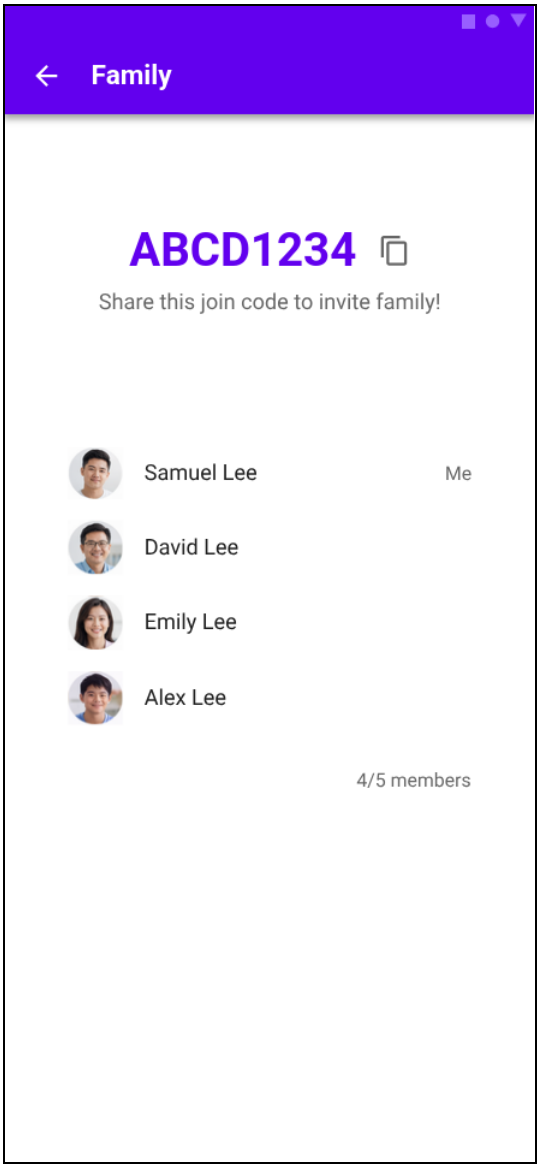

Parking Session
Parking time
Sat 07 Feb 2026 11:48:29 AM
Parking status
Ongoing
Accumulated fees
$12.34
Card value
$98.76
Driver
Samuel Lee
Car
Company Car
Location
NTU Carpark C
B1, Lot 123
Car plate
SXY1234Z
Parking duration: 3h 45min
END PARKING


Parking Session
Parking time
Sat 07 Feb 2026 11:48:29 AM
Parking status
Ongoing
Accumulated fees
$12.34
Card value
$98.76
Parking fee structure
Mon-Fri: $1.20 for 1st hr; $0.60 for sub. ½ hr or part thereof.
OK
B1, Lot 123
Car plate
SXY1234Z
Parking duration: 3h 45min
END PARKING

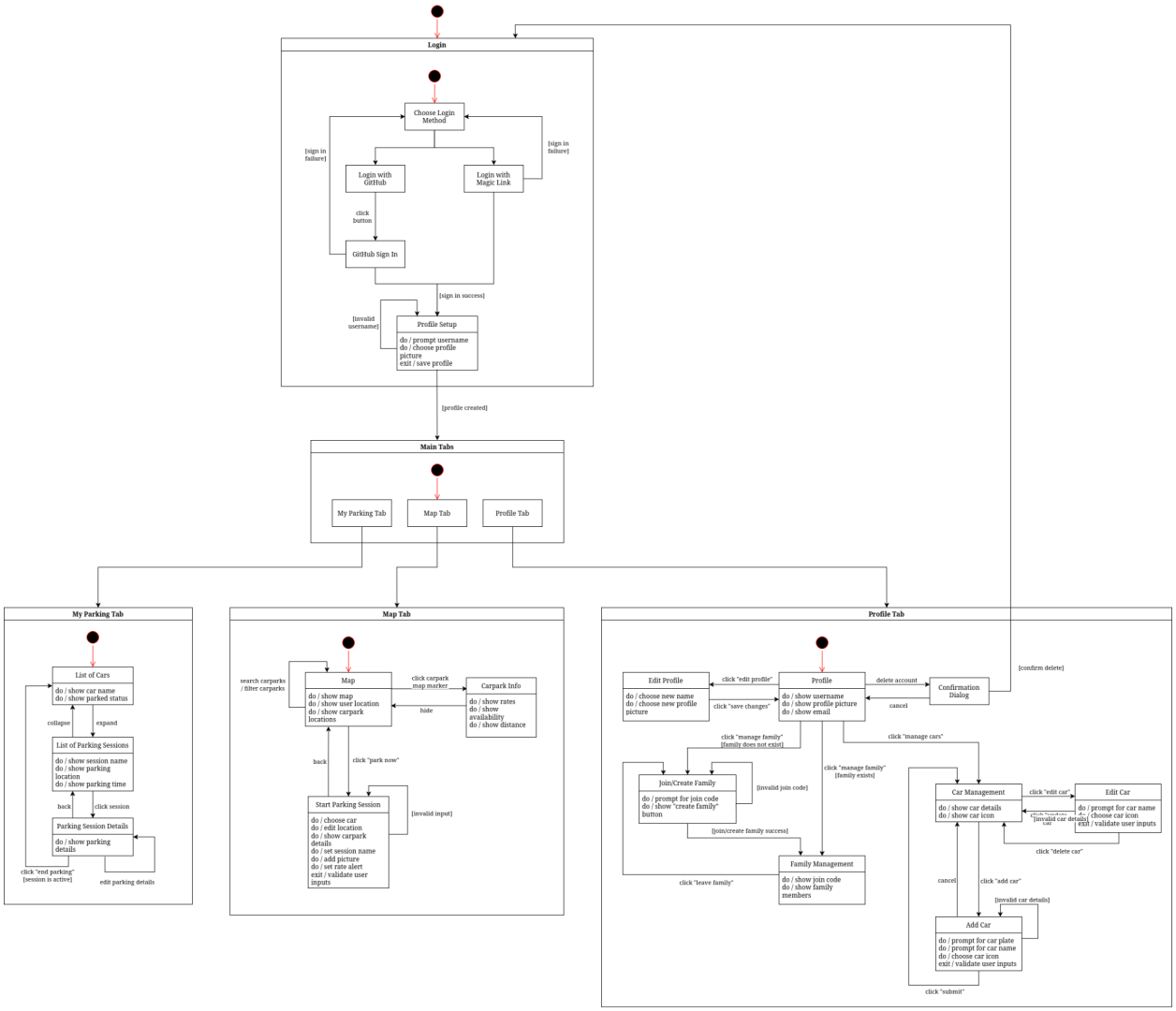
6.5 Profile


Profile
Samuel Lee
DELETE ACCOUNT
EDIT PROFILE
Username
Samuel Lee
Email
samuel.lee@gmail.com
Cars
Family
My Parking
Map
Profile

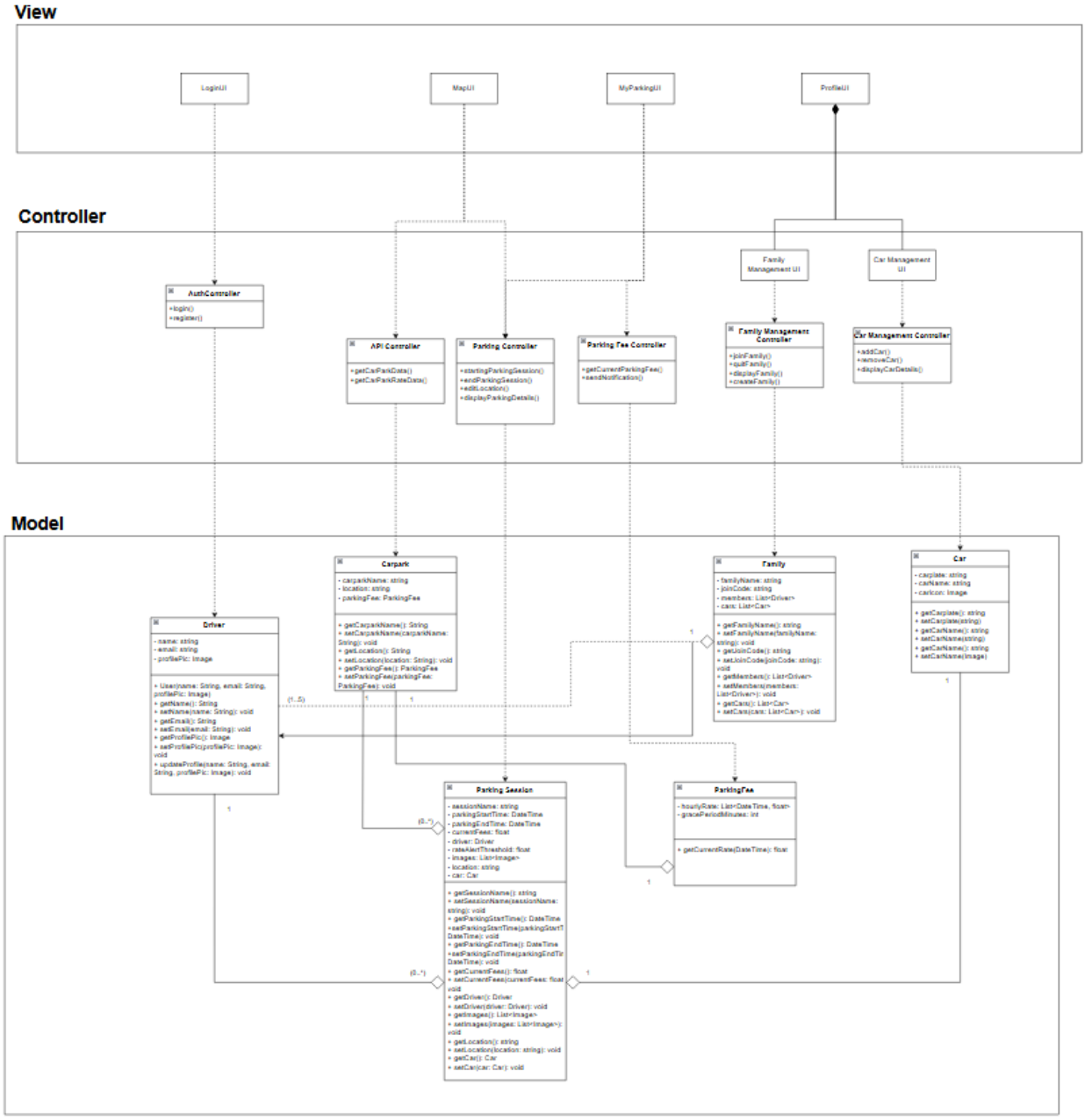

Cars
Company Car
SXY1234Z
Personal Car
SZX2345Y
Family Car
SYZ3456X
Friend's Car
SZY4567X
+ ADD CAR


Family
ABCD1234
Share this join code to invite family!
Samuel Lee
Me
David Lee
Emily Lee
Alex Lee
4/5 members

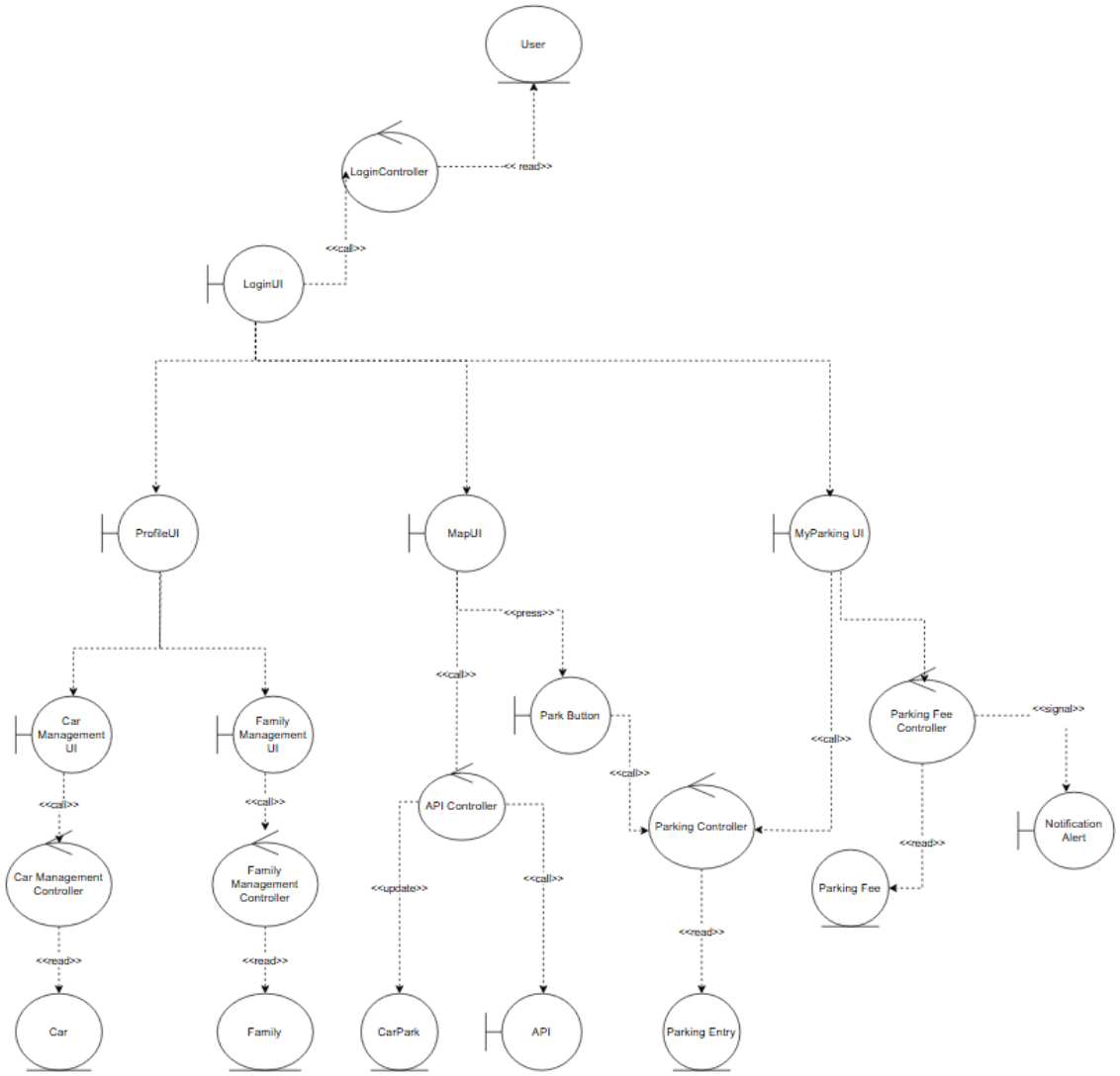
7. Dialogue Map



8. Entity Class Diagram



9. Stereotype Class Diagram


User
LoginController
<< read>>
<<call>>
LoginUI
ProfileUI
MapUI
MyParking UI
<<press>>
<<call>>
Car Management UI
Family Management UI
Park Button
Parking Fee Controller
<<signal>>
<<call>>
<<call>>
<<call>>
<<call>>
API Controller
Parking Controller
Notification Alert
Car Management Controller
Family Management Controller
<<read>>
Parking Fee
<<update>>
<<call>>
<<read>>
<<read>>
<<read>>
Car
Family
CarPark
API
Parking Entry